

VectorSearch

A vector database, built from scratch in C++

HNSW graph index

int8 quantization

Drogon REST API

RocksDB persistence

OpenTelemetry

Docker / Fly.io

Design Document · single-node architecture

1. Overview

VectorSearch is an approximate nearest neighbor (ANN) engine wrapped as a standalone database service. It stores high-dimensional vectors with attached JSON payloads, and answers "find the k most similar vectors to this query, optionally filtered by payload" over an HTTP API.

The hard core - the HNSW graph index and int8 scalar quantization - is written from scratch with no third-party ANN library. The surrounding database layer (API, collections, payload filtering, persistence, observability) turns that engine into a deployable service, in the same spirit as Qdrant or FAISS-server.

Goal 1

Fast, accurate ANN search over high-dimensional vectors.

Goal 2

A real benchmark proving recall and throughput against FAISS / hnswlib / Qdrant.

Goal 3

A clean single-node service: REST API, durable storage, metrics, one-command deploy.

2. System architecture

Requests enter through the Drogon HTTP layer. The Collection Manager owns one index per collection; a search runs the HNSW graph, refines the shortlist with exact distances on the quantized vectors, then applies the payload filter. Writes go to a WAL and periodic snapshots; payloads and metadata live in RocksDB. Every request is traced through OpenTelemetry.

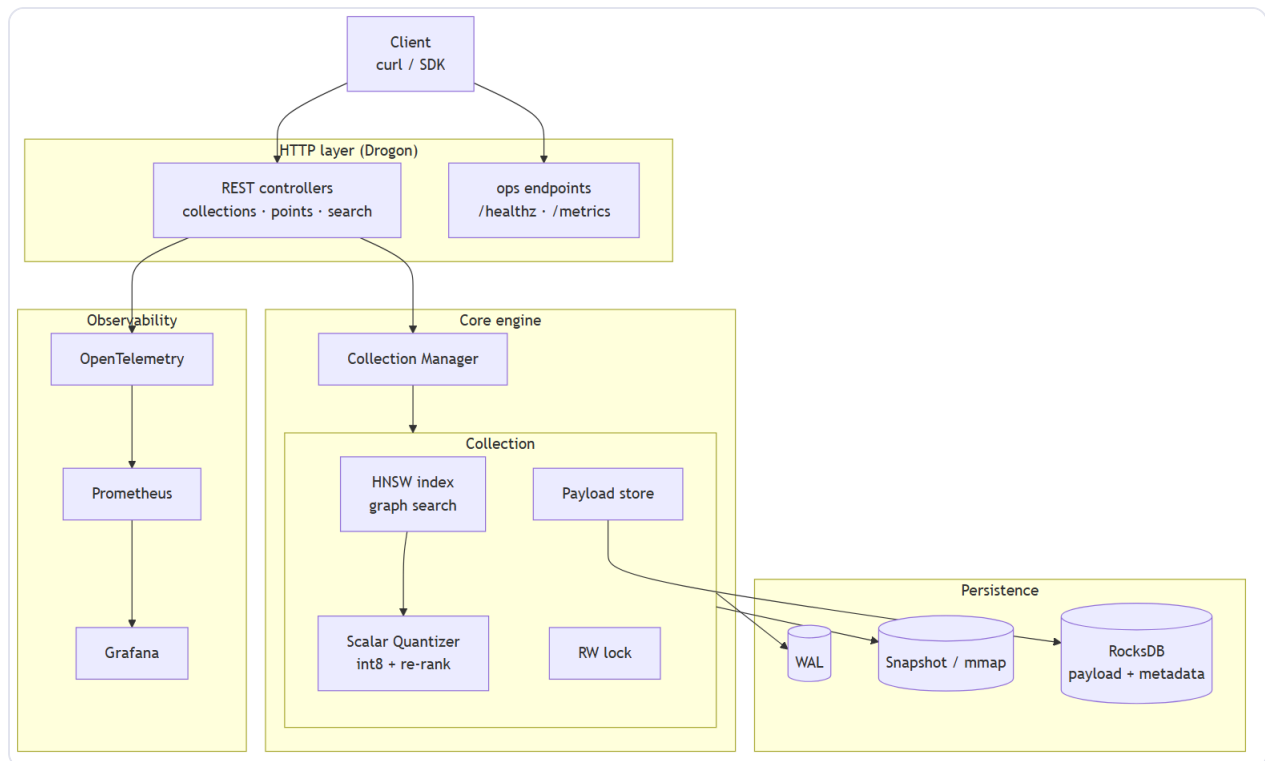


Figure 1 - Component architecture of the VectorSearch service.

3. Search request flow

A search over-fetches candidates from the graph (k times an over-fetch factor), re-ranks them with exact distances so the returned order is accurate, and only then filters by payload. Over-fetching before filtering keeps recall high when filters are selective.

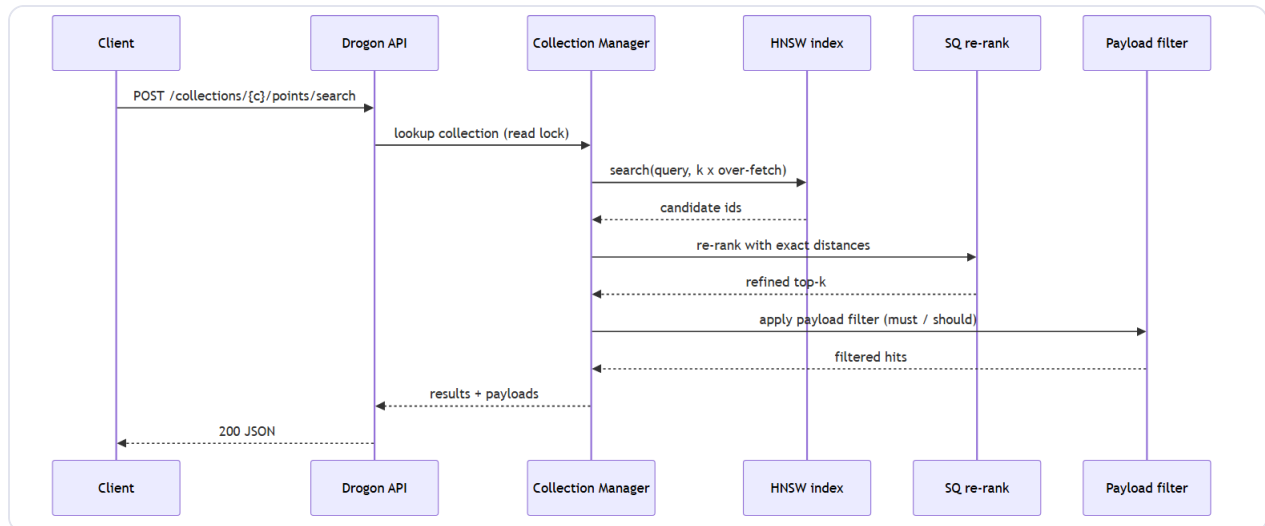


Figure 2 - Lifecycle of a single search request.

4. Data model

ENTITY	FIELDS	NOTES
Collection	name, dim, metric, index params	One HNSW index + payload store, guarded by a read/write lock.
Point	id (uint64), vector (f32[dim]), payload (JSON)	The unit of storage and retrieval.
Payload	arbitrary key/value JSON	Used for hybrid filtering during search.
Neighbor	id, distance, payload	A single search result, sorted closest first.

5. REST API

METHOD	PATH	PURPOSE
PUT	<code>/collections/{name}</code>	Create a collection (dim, metric, params)
GET	<code>/collections/{name}</code>	Collection info + point count
PUT	<code>/collections/{name}/points</code>	Upsert points (id, vector, payload)
POST	<code>/collections/{name}/points/search</code>	k-NN search with optional payload filter
DELETE	<code>/collections/{name}/points/{id}</code>	Delete a point
GET	<code>/healthz</code> · <code>/metrics</code>	Liveness probe · Prometheus metrics

The contract is published as an OpenAPI spec so a typed client SDK can be generated.

6. Persistence

Durability is layered so a crash never loses an acknowledged write and restart is fast:

- **Write-ahead log (WAL)** - every upsert/delete is appended before it is acknowledged; replayed on startup to recover the tail.
- **Snapshots** - the index and vectors are flushed to a compact binary file periodically, truncating the WAL.
- **RocksDB** - payloads and collection metadata live in an embedded key/value store (the same engine Qdrant uses), so lookups and updates are durable without bespoke code.
- **mmap** - large vector snapshots are memory-mapped so an index bigger than RAM still loads.

7. Build status & roadmap

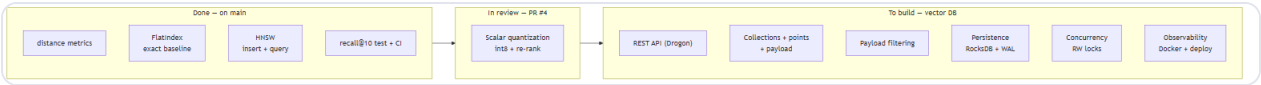


Figure 3 - What exists today versus the database layer still to build.

MILESTONE	SCOPE	STATUS
Engine	distance metrics, FlatIndex, HNSW insert + query, recall test, CI	done
Quantization	scalar int8 codec + exact re-rank pass	in review
M1	Drogon service skeleton + <code>/healthz</code>	to build
M2	collections + points + payload, core REST API	to build
M3	payload filtering (hybrid search)	to build
M4	persistence: RocksDB + WAL + snapshot recovery	to build
M5	concurrency: per-collection RW locks	to build
M6	observability + Docker + Fly.io deploy	to build

8. Tech stack

LAYER	CHOICE	WHY
Engine	C++17, from scratch	full control over the data structures and performance
HTTP	Drogon (async C++)	fast, modern, one language with the engine
Storage	RocksDB	battle-tested embedded KV, what real databases build on
Observability	OpenTelemetry + Prometheus + Grafana	standard, vendor-neutral telemetry
Delivery	Docker + Fly.io	reproducible build, one-command edge deploy
Contract	OpenAPI (REST)	generated clients, clear API surface